

Phase	Module / File	What to Code (Features)	Key "Flex" / Interview Talking Point	Time Needed
Phase 1	math.py	• Initialize an arbitrary n-dimensional list • Custom <code>__repr__</code> for clean printing • Magnitude (<code>mag()</code>), Dot Product (<code>dot()</code>) or overloading <code>@</code> , and Angle calculation.	• Use <code>__slots__</code> instead of <code>__dict__</code> to demonstrate memory optimization.	3 - 4 Hours(2 days)
Phase 2	<code>matrix.py</code> (Part 1)	• 2D list initialization • Shape checking and NumPy-style indexing/slicing (<code>__getitem__</code>) • Transpose property (<code>T</code> using <code>@property</code>) • Matrix-Matrix and Matrix-Vector multiplication (<code>__matmul__</code>).	• Enforce strict dimensional validation checks before performing any multiplication.	4 - 6 Hours(3 days)
Phase 3	<code>matrix.py</code> (Part 2)	• Elementary row operations (swap, scale, row addition) • Reduced Row Echelon Form (<code>ref()</code>) • Forward and Backward Substitution algorithms to solve triangular systems.	• Partial Pivoting in RREF to handle zero-diagonals • Use an epsilon tolerance threshold (like <code>1e-8</code>) instead of checking <code>== 0</code> directly.	6 - 8 Hours(4-5 days)
Phase 4	<code>matrix.py</code> (Part 3)	• Characteristic polynomial calculation or an iterative algorithm (like Power Iteration/QR Algorithm) to isolate Eigenvalues and Eigenvectors.	• Explain that you implemented an iterative eigensolver because solving roots of high-degree polynomials analytically is impossible (Abel-Ruffini theorem).	8 - 12 Hours(1 week)
Phase 5	decomposition.py	• PLU Decomposition (<code>PLU(L,U)</code>) • QR Decomposition via Modified Gram-Schmidt • SVD (<code>listVT</code>) built using your Phase 4 eigensolver.	• Choose Modified Gram-Schmidt over Classical Gram-Schmidt because it is drastically more numerically stable against rounding errors.	12 - 18 Hours(+ 1-2 weeks)
Phase 6	predict.py	• A <code>LinearRegression</code> class with <code>fit(X, y)</code> and <code>predict(X)</code> .	• The Ultimate Flex: You solve the Normal Equation ($XTX\hat{y}=XTY$) using your QR decomposition and Backward Substitution, completely avoiding the unstable matrix inverse.	4 - 6 Hours(2-3 days)
Phase 7	<code>tests/</code> & <code>ADocument</code>	• A test suite using Python's native <code>unittest</code> module • Compare your outputs against <code>numpy.linalg</code> to prove accuracy • Write a comprehensive GitHub README.md.	• Proves production-grade engineering discipline. You aren't just claiming the math works; you have automated tests verifying it.	4 - 6 Hours(3 days)